CSE513 Assignment 1 - CPU Scheduling Simulator

Public website URL: PENDING OWNER-AUTHENTICATED STREAMLIT COMMUNITY CLOUD DEPLOYMENT

Deployment action required: push the production commit to the configured GitHub repository, create a Community Cloud app from `main/app.py`, make it public, and verify Workload A/FCFS and Workload B/RR q=2 in a private browser session. No URL is fabricated in this report.

Prepared: 24 October 2026 Implementation: Python 3.11, Streamlit, Plotly Repository: `https://github.com/MesutHakanTaskiner/GTU-OS-Project`

1. Objective and model

This project is a deterministic browser-based simulator for FCFS, non-preemptive SJF, preemptive SRTF, and Round Robin. It emphasizes scheduling correctness, reproducible calculations, and classroom explainability.

The model contains exactly one CPU, one CPU burst per process, no I/O, and zero-time context switches. Arrival and execution times are integers. Each timeline interval is half-open: `[start,end)`. The CPU cannot idle while a ready process exists. Process identifiers are unique, and the table row order is retained as immutable `input_index`.

This simulator models scheduling policies. It does **not** reproduce the actual Ubuntu scheduler.

The professor's file `CSE513-Assignment-1.pdf` was not present in the repository during development. The supplied `AGENTS.md` requirements were therefore used as the working specification; the original PDF must be added and checked before academic submission.

2. Algorithms and deterministic tie rules

- **FCFS:** choose the earliest-arriving ready process and run it to completion.
- **SJF:** when the CPU is free, choose the ready process with the smallest original burst and run it non-preemptively. A future short job is never selected early.
- **SRTF:** choose the smallest remaining burst. Preempt a running process only when another ready process is strictly shorter. Equal remaining time keeps the current process.
- **Round Robin:** use a FIFO ready queue. New arrivals append at the tail. At a quantum boundary, arrivals at that exact time are enqueued before the unfinished running process. Each slice remains a distinct timeline segment.

When a new selection has an otherwise equal policy key, earlier arrival wins, followed by earlier original input index. Initial and intermediate idle gaps are explicit `IDLE` segments.

3. Implementation architecture

The `scheduler` package is pure and deterministic. Frozen dataclasses represent process specifications, execution segments, metrics, and complete results. `simulate(processes, algorithm, quantum=None)` validates immutable inputs, dispatches one policy function, derives metrics from the finished timeline, and returns a result with a decision trace.

The Streamlit layer only manages editing, presets, state freshness, presentation, and error messages. Plotly renders the timeline. A separate verifier checks burst accounting, arrival safety, non-overlap and continuity, nonnegative metrics, and the work-conserving rule. Thus the UI does not contain scheduling decisions.

Flow: validated table -> pure policy scheduler -> execution segments -> metrics -> invariant verifier -> Plotly/table/trace.

4. Required worked examples

The following hand results were derived before consulting simulator output. `C` is completion, `T=C-arrival`, `W=T-burst`, and `R=first_start-arrival`.

4.1 Workload A input

Process	Arrival	Burst
P1	0	8
P2	1	4
P3	2	2

Process	Arrival	Burst
P4	4	1

4.2 Workload A - FCFS

Manual decisions: at t=0 only P1 is ready, so it runs through t=8. P2, P3, and P4 arrive while P1 runs. Their arrival order is P2, P3, P4, so they execute in that order.

Manual timeline: `P1[0,8) -> P2[8,12) -> P3[12,14) -> P4[14,15).`

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	8	12	11	7	7
P3	2	2	12	14	12	10	10
P4	4	1	14	15	11	10	10

Manual sums: `T=8+11+12+11=42`, `W=0+7+10+10=27`, `R=0+7+10+10=27`. Averages: T=10.50, W=6.75, R=6.75. Simulator output matched every interval, integer metric, and average; discrepancy: none.

4.3 Workload A - SJF

Manual decisions: P1 is the only ready process at t=0 and SJF is non-preemptive, so P1 runs to t=8. All other processes are then ready. Their burst order is P4(1), P3(2), P2(4).

Manual timeline: `P1[0,8) -> P4[8,9) -> P3[9,11) -> P2[11,15).`

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	8	8	0	0
P2	1	4	11	15	14	10	10
P3	2	2	9	11	9	7	7
P4	4	1	8	9	5	4	4

Manual sums: `T=8+14+9+5=36`, `W=0+10+7+4=21`, `R=0+10+7+4=21`. Averages: T=9.00, W=5.25, R=5.25. Simulator comparison: exact match; discrepancy: none.

4.4 Workload A - SRTF

Manual decisions: P1 starts at t=0. At t=1, P2(4) is strictly shorter than P1's remaining 7, so P2 preempts. At t=2, P3(2) is strictly shorter than P2's remaining 3, so P3 preempts and finishes at t=4. P4(1) is then shortest and finishes at t=5. P2(3) runs to t=8, followed by P1(7) to t=15.

Manual timeline: `P1[0,1) -> P2[1,2) -> P3[2,4) -> P4[4,5) -> P2[5,8) -> P1[8,15).`

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	1	8	7	3	0
P3	2	2	2	4	2	0	0
P4	4	1	4	5	1	0	0

Manual sums: `T=15+7+2+1=25`, `W=7+3+0+0=10`, `R=0`. Averages: T=6.25, W=2.50, R=0.00. Simulator comparison: exact match; discrepancy: none.

4.5 Workload A - Round Robin, q=2

Manual decisions: P1 runs 0-2. P2 arrives at 1 and P3 at the boundary t=2; both enter before P1 is re-enqueued, giving P2,P3,P1. P2 runs 2-4. P4 arrives at boundary t=4 before P2 re-enters, giving P3,P1,P4,P2. Subsequent slices follow FIFO. P1's final two adjacent slices remain distinct.

Manual timeline: `P1[0,2) -> P2[2,4) -> P3[4,6) -> P1[6,8) -> P4[8,9) -> P2[9,11) -> P1[11,13) -> P1[13,15)`.

Process	Arrival	Burst	First start	C	T	W	R
P1	0	8	0	15	15	7	0
P2	1	4	2	11	10	6	1
P3	2	2	4	6	4	2	2
P4	4	1	8	9	5	4	4

Manual sums: T=15+10+4+5=34, W=7+6+2+4=19, R=0+1+2+4=7. Averages: T=8.50, W=4.75, R=1.75. Simulator comparison: exact match, including separate final P1 slices; discrepancy: none.

4.6 Workload B input

Process	Arrival	Burst
P1	2	3
P2	2	1
P3	8	2
P4	9	1

4.7 Workload B - FCFS

Manual decisions: no process exists until t=2. P1 and P2 arrive together, so input order selects P1 then P2. Both finish at t=6; the CPU idles until P3 arrives at 8. P3 runs non-preemptively, then P4.

Manual timeline: `IDLE[0,2) -> P1[2,5) -> P2[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	2	5	3	0	0
P2	2	1	5	6	4	3	3
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Manual sums: T=3+4+2+2=11, W=0+3+0+1=4, R=0+3+0+1=4. Averages: T=2.75, W=1.00, R=1.00. Simulator comparison: exact match; discrepancy: none.

4.8 Workload B - SJF

Manual decisions: after IDLE 0-2, both P1(3) and P2(1) are ready, so P2 runs first, followed by P1. The CPU idles 6-8. P3 starts at 8; P4 arrives at 9, but non-preemptive SJF lets P3 finish before P4.

Manual timeline: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	3	6	4	1	1

Process	Arrival	Burst	First start	C	T	W	R
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Manual sums: T=4+1+2+2=9, W=1+0+0+1=2, R=1+0+0+1=2. Averages: T=2.25, W=0.50, R=0.50. Simulator comparison: exact match; discrepancy: none.

4.9 Workload B - SRTF

Manual decisions through t=8 match SJF. P3 starts at 8. At t=9, P3 has one unit remaining and P4 arrives with one unit. Because equality cannot preempt the running process, P3 keeps the CPU until t=10; P4 then runs.

Manual timeline: `IDLE[0,2) -> P2[2,3) -> P1[3,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	3	6	4	1	1
P2	2	1	2	3	1	0	0
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Manual sums and averages are the same as SJF: T=9/4=2.25, W=2/4=0.50, R=2/4=0.50. Simulator comparison: exact match and P3 remained a merged 8-10 segment; discrepancy: none.

4.10 Workload B - Round Robin, q=2

Manual decisions: after IDLE 0-2, P1 enters before P2 by input order. P1 uses a two-unit slice, P2 runs one unit and completes, then P1 completes its final unit. The CPU idles 6-8. P3 completes within one slice while P4 waits, then P4 runs.

Manual timeline: `IDLE[0,2) -> P1[2,4) -> P2[4,5) -> P1[5,6) -> IDLE[6,8) -> P3[8,10) -> P4[10,11)`.

Process	Arrival	Burst	First start	C	T	W	R
P1	2	3	2	6	4	1	0
P2	2	1	4	5	3	2	2
P3	8	2	8	10	2	0	0
P4	9	1	10	11	2	1	1

Manual sums: T=4+3+2+2=11, W=1+2+0+1=4, R=0+2+0+1=3. Averages: T=2.75, W=1.00, R=0.75. Simulator comparison: exact match; discrepancy: none.

5. Student-designed targeted cases

5.1 Initial and intermediate idle time

Input: I1=(arrival 2, burst 2), I2=(arrival 6, burst 1), tested with FCFS. Expected and actual timeline: `IDLE[0,2) -> I1[2,4) -> IDLE[4,6) -> I2[6,7)`. I1 has T=2,W=0,R=0; I2 has T=1,W=0,R=0. This verifies both idle forms and confirms that idle time is represented rather than skipped.

5.2 Simultaneous arrivals and equal bursts

Input order: T1=(0,3), T2=(0,1), T3=(0,1), tested with SJF. Expected and actual timeline: `T2[0,1) -> T3[1,2) -> T1[2,5)`. T1 has T=5,W=2,R=2; T2 has T=1,W=0,R=0; T3 has T=2,W=1,R=1. This proves that equal shortest bursts use original input order.

5.3 SRTF strict and equal-remaining behavior

Input: E1=(0,5), E2=(1,4), E3=(2,1). At t=1, E1 has four units left, equal to E2, so E1 must continue. At t=2, E1 has three left and E3 has one, so E3 must preempt. Expected and actual timeline: `E1[0,2) -> E3[2,3) -> E1[3,6) -> E2[6,10)`. Metrics: E1 T=6,W=1,R=0; E2 T=9,W=5,R=5; E3 T=1,W=0,R=0.

5.4 Additional RR boundary regression

Input B1=(0,4), B2=(2,1), q=2. Expected and actual timeline: `B1[0,2) -> B2[2,3) -> B1[3,5)`. This proves that B2's boundary arrival is enqueued before unfinished B1. A separate single-process test confirms that `Solo[0,2)`, `Solo[2,4)`, and `Solo[4,5)` remain three distinct slices.

All targeted cases passed the five independent invariants.

6. Analysis questions

6.1 Minimum average waiting and response for Workload A

SRTF is minimum for both measures among the four required runs: average waiting 2.50 and average response 0.00. It immediately starts P1 at t=0 and each later arrival P2, P3, and P4 at its arrival because each is strictly shorter than the current remainder. FCFS averages 6.75/6.75, SJF 5.25/5.25, and RR q=2 4.75/1.75 for waiting/response. SRTF moves short jobs forward without delaying their first service, at the cost of completing long P1 last.

6.2 RR quantum 1, 2, and 4 on Workload A

Quantum	Avg. turnaround	Avg. waiting	Avg. response
1	8.50	4.75	0.75
2	8.50	4.75	1.75
4	9.25	5.50	3.75

Quantum 1 improves average response because P2, P3, and P4 receive their first turn sooner. It does not improve turnaround or waiting relative to q=2: extra rotation changes when partial work occurs but yields the same completion-based averages here. Smaller quantum therefore does not uniformly improve metrics. In a real system it would also increase context-switch overhead, although this simulator assigns context switches zero time.

6.3 When SJF and SRTF are identical

They are identical when no arrival during a running job has a burst strictly shorter than that job's remaining time, and both select the same ready job whenever the CPU becomes free. Workload B is a concrete example. At t=9, P4 arrives with burst 1 while P3 has remaining 1; equality does not preempt, so both policies produce P3[8,10) then P4[10,11).

6.4 Why real systems estimate CPU bursts

This simulator receives the entire burst as input, which makes SJF/SRTF selection exact. A real operating system cannot know how long a program will compute before it blocks, exits, or is interrupted; that behavior depends on data, branches, cache effects, and external events. Practical schedulers therefore use history, feedback, priorities, fairness, and estimators rather than an oracle for the upcoming CPU burst.

6.5 Starvation

SJF can postpone a long ready job whenever shorter jobs are available at each selection point. SRTF can repeatedly preempt a long job if a sustained stream of shorter remaining workloads arrives. Thus both can starve long jobs in an open-ended system without aging or fairness controls. These supplied finite workloads eventually stop receiving arrivals, so every process completes; finite completion here does not remove the general starvation risk.

6.6 Real implementation mistake and correction

The first Streamlit smoke test used `AppTest.from_file("app.py")` inside `tests/test_app.py`. Streamlit resolves relative paths against the calling test file, so pytest failed with `FileNotFoundError` for `tests/app.py`. The correction was to resolve the repository root explicitly with `Path(__file__).resolve().parents[1] / "app.py"`. Rerunning the suite then passed all 61 tests. This was not hidden or rewritten as a scheduling anecdote; it was the actual failure encountered during development.

7. Validation and test strategy

Validation rejects empty or duplicate IDs, negative arrival, nonpositive burst/quantum, decimal floats, booleans, strings posing as numbers, NaN, malformed rows, unknown algorithms, duplicate input indices, over 100 processes, individual values over 100,000, and total burst over 1,000,000.

The tests combine independently written exact timelines/metrics with policy regressions and property-style invariants. Required workloads A and B run under all four algorithms. Additional coverage includes SJF non-preemption, SRTF equal-time continuation, RR exact-boundary arrival order, consecutive RR slices, simultaneous arrivals, completion at an arrival boundary, and initial/middle idle intervals.

Commands and actual results on Python 3.11.0:

```
python -m pytest -q
61 passed in 1.48s

streamlit run app.py --server.headless true --server.port 8501
GET /_stcore/health -> HTTP 200, ok
GET / -> HTTP 200
```

The visual browser surface was unavailable to the automated environment, so a screenshot-based desktop inspection was not claimed. Streamlit's application test executed the default Workload A/FCFS flow and verified its three displayed averages without an exception.

8. Interface and usability

The single-page interface provides Workload A/B buttons, three targeted presets, dynamic row editing, contextual algorithm explanations, an RR-only quantum control, precise errors, a primary run action, metric cards, the full eight-column table, a Plotly CPU lane, accessible text order, cross-policy comparison, and a decision trace. Results store a fingerprint of the simulated table/policy/quantum; edits trigger an explicit stale-result warning until rerun.

Idle intervals use grey. Process colors are stable within a run. RR uses white three-pixel borders and per-slice hover labels so adjacent slices never visually disappear into one block. Every timeline boundary appears as an integer tick and guide line.

9. AI assistance disclosure

OpenAI Codex assisted with requirement organization, Python/Streamlit implementation, test drafting, documentation, and report formatting. Scheduling expectations in `report/manual_expectations.md` were written before executing the implementation and were then compared with simulator output. The test suite, invariant verifier, Streamlit smoke test, HTTP health check, and rendered PDF review were used to verify generated work.

One AI-produced test initially used the wrong relative path for Streamlit's `AppTest`; pytest rejected it, and the path was corrected to an absolute repository-root-derived path. A separate Arrow warning in the policy comparison table was

corrected by giving the Quantum column a consistent string type. Suggestions were accepted only after executable checks.

10. Local use and deployment

Create a Python 3.11 virtual environment, install `requirements-dev.txt`, run `python -m pytest -q`, then run `streamlit run app.py`. The production runtime uses only the pinned packages in `requirements.txt`.

For Community Cloud, authenticate at `share.streamlit.io`, choose repository `MesutHakanTaskiner/GTU-OS-Project`, branch `main`, and entrypoint `app.py`; select Python 3.11 and deploy without secrets. Make the app public, open its assigned `.streamlit.app` URL in a private window, and verify Workload A/FCFS plus Workload B/RR q=2. Record that real URL on page 1 only after verification.

11. Limitations and final status

The simulation intentionally excludes I/O bursts, multicore behavior, nonzero context-switch cost, priorities, and the Linux Completely Fair Scheduler. It is a policy simulator, not a kernel emulator. Public deployment remains dependent on authenticated owner action; therefore the website URL is correctly marked pending rather than invented. The authoritative professor PDF must also be compared once supplied.